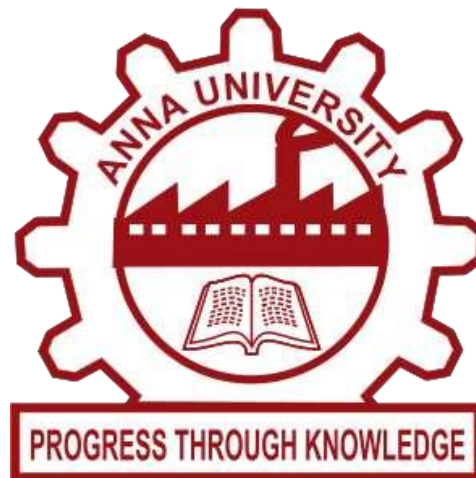


# **UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL**

**(ANNA UNIVERSITY CONSTITUENT COLLEGE)**

**KONAM, NAGERCOIL – 629 004**



## **RECORD NOTE BOOK**

**CCS375-WEB TECHNOLOGIES LABORATORY**

**Register No        :**

**Name                :**

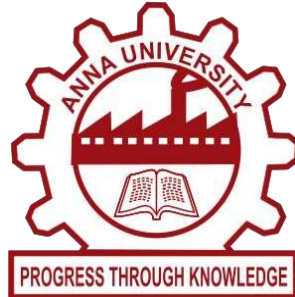
**Year/Semester    :**

**Department       :**

# UNIVERSITY COLLEGE OF ENGINEERING NAGERCOIL

(ANNA UNIVERSITY CONSTITUENT COLLEGE)

KONAM, NAGERCOIL – 629 004



**Register No:**

*Certified that, this is the bonafide record of work done by  
Mr./Ms. .... of V Semester  
in Computer Science and Engineering of this college, in the **CCS375-Web  
Technologies Laboratory** during academic year 2025-2026 in partial  
fulfillment of the requirements of the B.E Degree course of the Anna  
University Chennai.*

**Staff-in-charge**

**Head of the Department**

This record is submitted for the University Practical Examination  
held on .....

**Internal Examiner**

**External Examiner**

---

INDEX

[illegible]

| Exp No | Date | Title | Page | Sign |
|--------|------|-------|------|------|
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |
|        |      |       |      |      |

Date:

**Aim:**

To create a web page using HTML that embeds an image map, defines hotspots, and displays related information when the hotspots are clicked.

**Description:**

This experiment demonstrates how to create an interactive image map using HTML. An image map allows specific regions of an image—called hotspots—to be clickable, each linking to a different HTML page with relevant information. In this example, the map of India is used, and hotspots are defined for Tamil Nadu and Karnataka using the `area` tags. Each hotspot is assigned coordinates and a shape (circle or rectangle), and clicking on them navigates to a separate page displaying details about the selected state. This technique is useful for building visually engaging navigation systems in web applications.

**Algorithm:**

**Step 1:** Start

**Step 2:** Create an HTML file and load the image of the India map

**Step 3:** Use the `img` tag to define a named image map

**Step 4:** Use `area` tags to define clickable hotspots with shapes and coordinates

**Step 5:** Link each hotspot to a corresponding HTML page (e.g., `tamilnadu.html`, `karnataka.html`)

**Step 6:** Create separate HTML pages for each state with relevant information

**Step 7:** Save and run the HTML file in a browser to test hotspot functionality

**Step 8:** Stop

**Program:****main.html**

```
<head>
<BODY bgcolor="#gop6876cgdt5564ss">

<map name=indiamap>
<area shape="circle" coords="340,971,107,180" href="tamilnadu.html" alt="Tamilnadu">
<area shape="rect" coords="247,827,107,180" href="karnataka.html" alt="Karnataka">
</map>
</head>
</BODY>
```

</html>

## Karnataka.html

<!DOCTYPE html>

<html>

<head>

<title>Karnataka - India</title>

</head>

<body style="background-color: wheat;">

<h1 style="text-align: center;">Karnataka</h1>

<p>

Karnataka is a southern Indian state known for its tech capital, Bengaluru. It has a rich history with UNESCO sites like Hampi and vibrant cultural traditions. The state also features lush landscapes, from the Western Ghats to coastal beaches.

</p>

<ul>

<li>Districts: 30</li>

<li>Capital City: Bangalore</li>

<li>Largest City: Bangalore</li>

<li>Governor: Thawar Chand Gehlot</li>

<li>Chief Minister: Siddaramaiah</li>

<li>Population: 61,130,704</li>

<li>Tourist Spots: Gol Gumbaz, Mysore Palace, Keshava Temple</li>

</ul>

<a href="main.html">Back</a>

</body>

</html>

## Tamilnadu.html

<!DOCTYPE html>

<html>

<head>

<title>Tamil Nadu - India</title>

</head>

<body style="background-color: palegreen;">

<h1 style="text-align: center;">Tamil Nadu</h1>

<p>

Tamil Nadu is one of the 29 states of India. Its capital and largest city is Chennai. It lies in the southernmost part of the Indian Peninsula and is bordered by Puducherry, Kerala, Karnataka, and Andhra Pradesh.

</p>

<ul>

<li>Districts: 33</li>

<li>Capital City: Chennai</li>

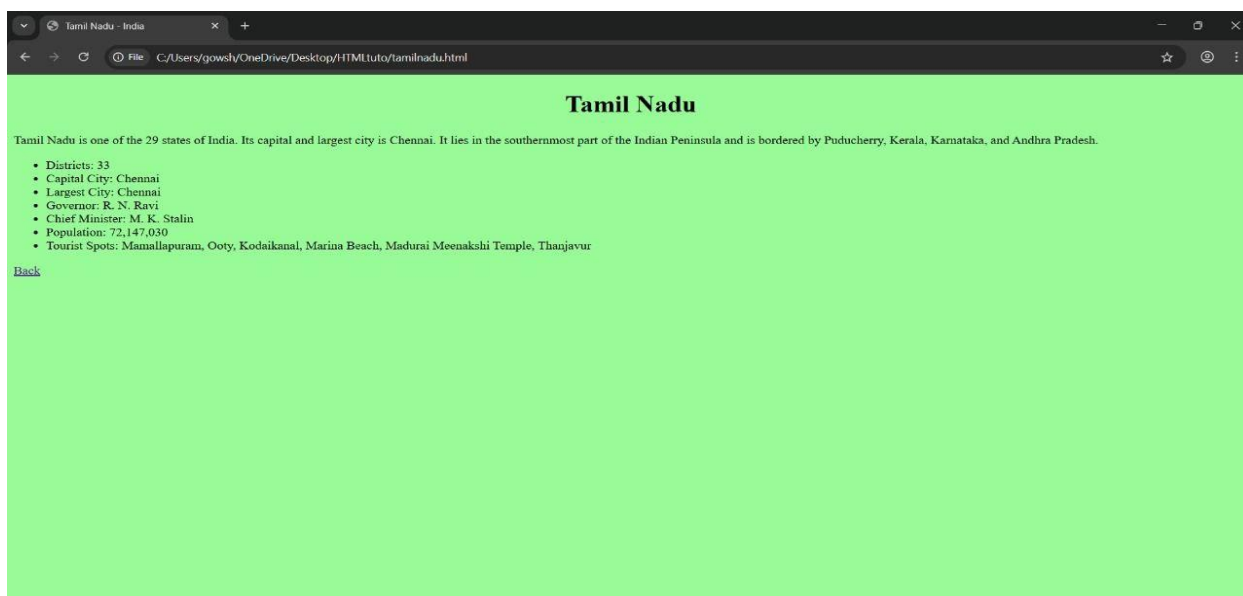
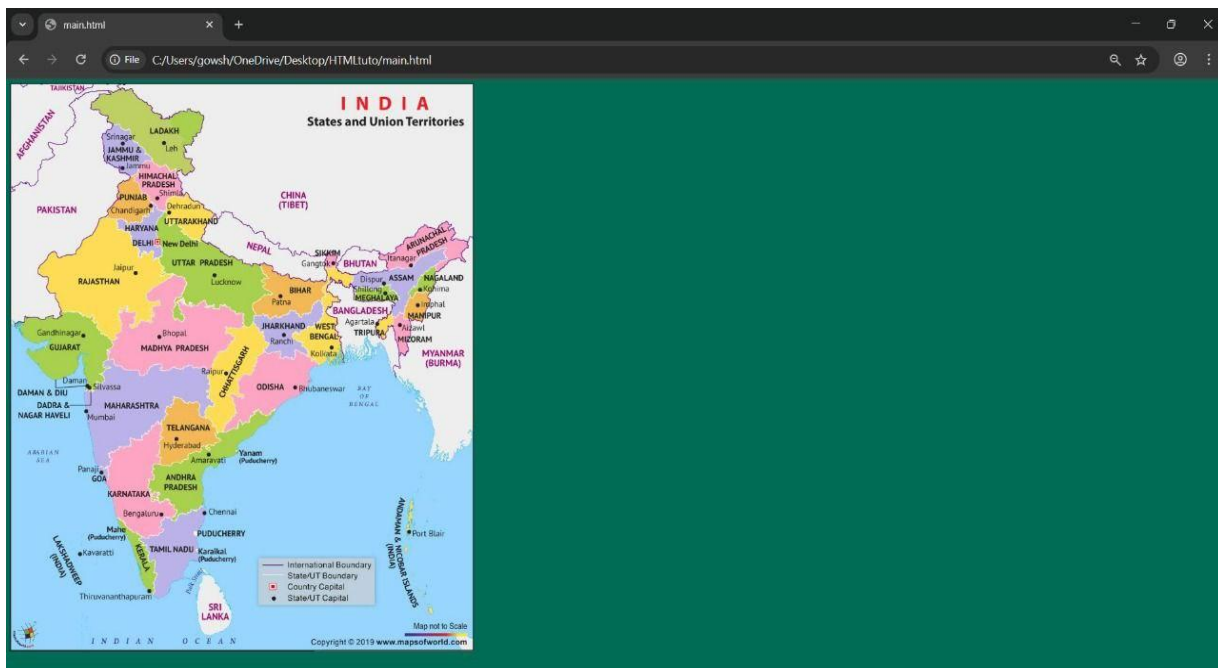
<li>Largest City: Chennai</li>

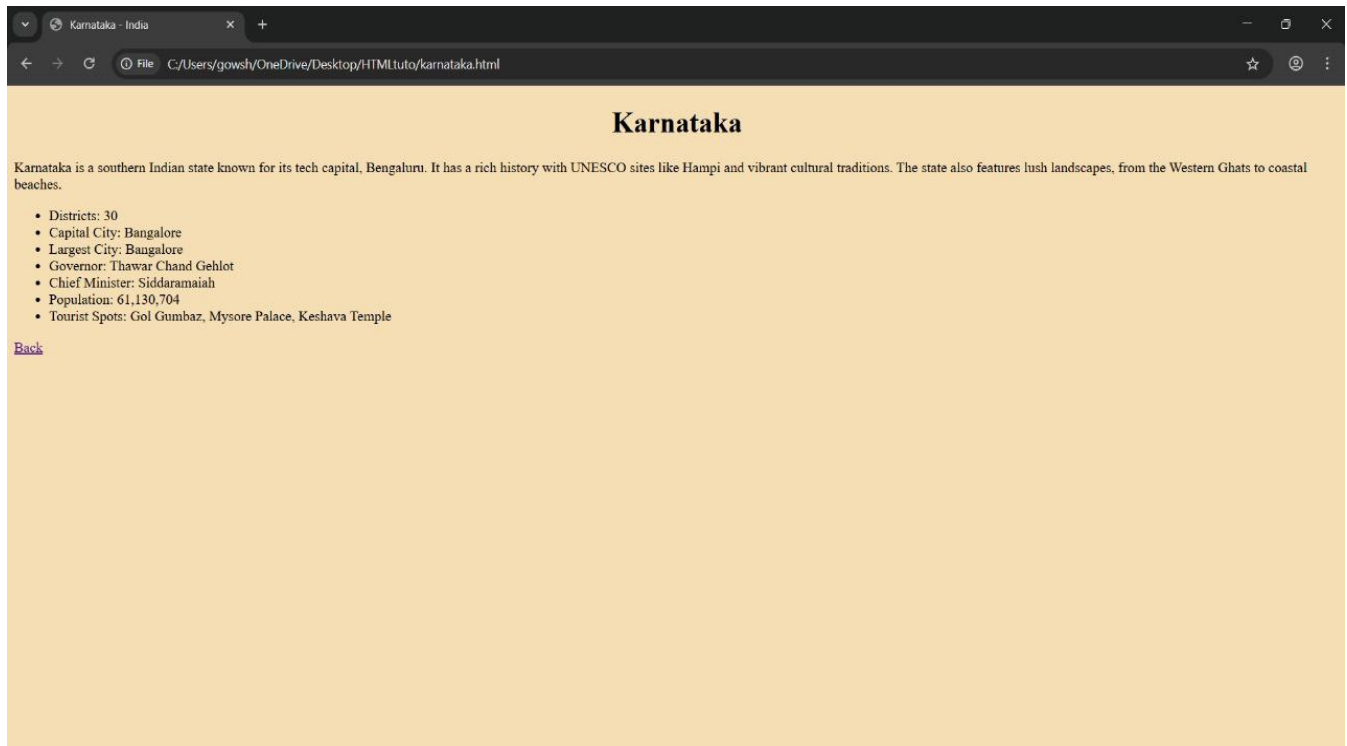
```

<li>Governor: R. N. Ravi</li>
<li>Chief Minister: M. K. Stalin</li>
<li>Population: 72,147,030</li>
<li>Tourist Spots: Mamallapuram, Ooty, Kodaikanal, Marina Beach, Madurai Meenakshi
Temple, Thanjavur</li>
</ul>
<a href="main.html">Back</a>
</body>
</html>

```

## OUTPUT:





## Result:

The web page was successfully created using HTML with an embedded image map. Hotspots were defined for Tamil Nadu and Karnataka, and clicking on each hotspot correctly navigated to a separate page displaying detailed information about the respective state.



|                  |                                                                           |                 |
|------------------|---------------------------------------------------------------------------|-----------------|
| <b>Ex. No: 2</b> | <b>Creation of Web Page Using All Types of<br/>Cascading Style Sheets</b> | <b>Page No:</b> |
| <b>Date:</b>     |                                                                           |                 |

### Aim:

To design a web page using all three types of Cascading Style Sheets (CSS): Inline, Internal, and External, to demonstrate styling techniques in web development.

### Description:

This experiment demonstrates the use of all three types of Cascading Style Sheets (CSS) in a single web page: **Inline**, **Internal**, and **External**. CSS is used to control the presentation and layout of HTML elements.

- **Inline CSS** is applied directly within an HTML tag using the style attribute.
- **Internal CSS** is defined within the tag inside the section of the HTML document.
- **External CSS** is written in a separate .css file and linked to the HTML using the tag.

In this experiment, a web page about flowers is styled using all three methods. The external stylesheet (style.css) controls the overall layout and link behavior, the internal stylesheet styles the paragraph, and inline styles are used for specific elements like the table and span text. This approach helps understand how different CSS types interact and affect the final appearance of a web page.

### Algorithm:

**Step 1:** Start

**Step 2:** Create an HTML file and define the basic structure using , , and tags

**Step 3:** Link an external CSS file using the tag inside the section

**Step 4:** Add internal CSS rules using the tag to style paragraph elements

**Step 5:** Apply inline CSS directly to specific HTML elements like and using the style attribute

**Step 6:** Create content including headings, paragraphs, and a list of flower links

**Step 7:** Save and run the HTML file in a browser to observe the styling effects

**Step 8:** Stop

### Program:

**style.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>FLOWERS</title>
```

```

<!-- External Style Sheet -->
<link rel="stylesheet" type="text/css" href="style.css">

<!-- Embedded Style Sheet -->
<style type="text/css">
  p {
    background-color: lightgrey;
    text-align: justify;
    margin: 2em 7em;
  }
</style>
</head>
<body id="body">
  <h1>FLOWER</h1>
  <p>
    <span style="font-weight: 200; font-size: x-large; font-family: fantasy;">Flower</span>
    sometimes known as a bloom or blossom, is the reproductive structure found in
flowering plants.
    The flower is God's finest workmanship in the world. It is his finest gift to mankind.
    We have seen flowers of many kinds and many colors. In India we see flowers like:
  </p>

  <!-- Inline Style Sheet -->
  <table style="margin: auto; text-align: center; padding: 3px;">
    <tr>
      <td align="left">
        <div class="div">
          <ul>
            <li><a href="https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcSTgtoKchwB7lfBAKx_U4cGrziXuzWn31UelQ&s
">Lily</a></li>
            <li><a href="https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcTqQi7iFH1j3qpXA4lxTPSaxI0NAX9PXNUyTw&
s">Lotus</a></li>
            <li><a href="https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcQCswEByOj8Z_aIeivnhuxdsw_OUthYAsUTNQ&
s">Rose</a></li>
            <li><a href="https://encrypted-
tbn0.gstatic.com/images?q=tbn:ANd9GcS0QpzKsDSXBxzcjQ5eh_7yuG6t-
DgedgM_iw&s">Jasmine</a></li>
          </ul>
        </div>
      </td>
    </tr>
  </table>
</body>

```

</html>

## Style.css

```
h1, h2 {  
    text-decoration: underline;  
    font-style: italic;  
    text-align: center;  
}
```

```
#body {  
    background-color: tan;  
    border: 2px dotted red;  
    text-align: center;  
}
```

```
.div {  
    border: 2px solid peru;  
    padding: 10px;  
}
```

```
* {  
    letter-spacing: 1px;  
}
```

```
a:link {  
    color: black;  
}
```

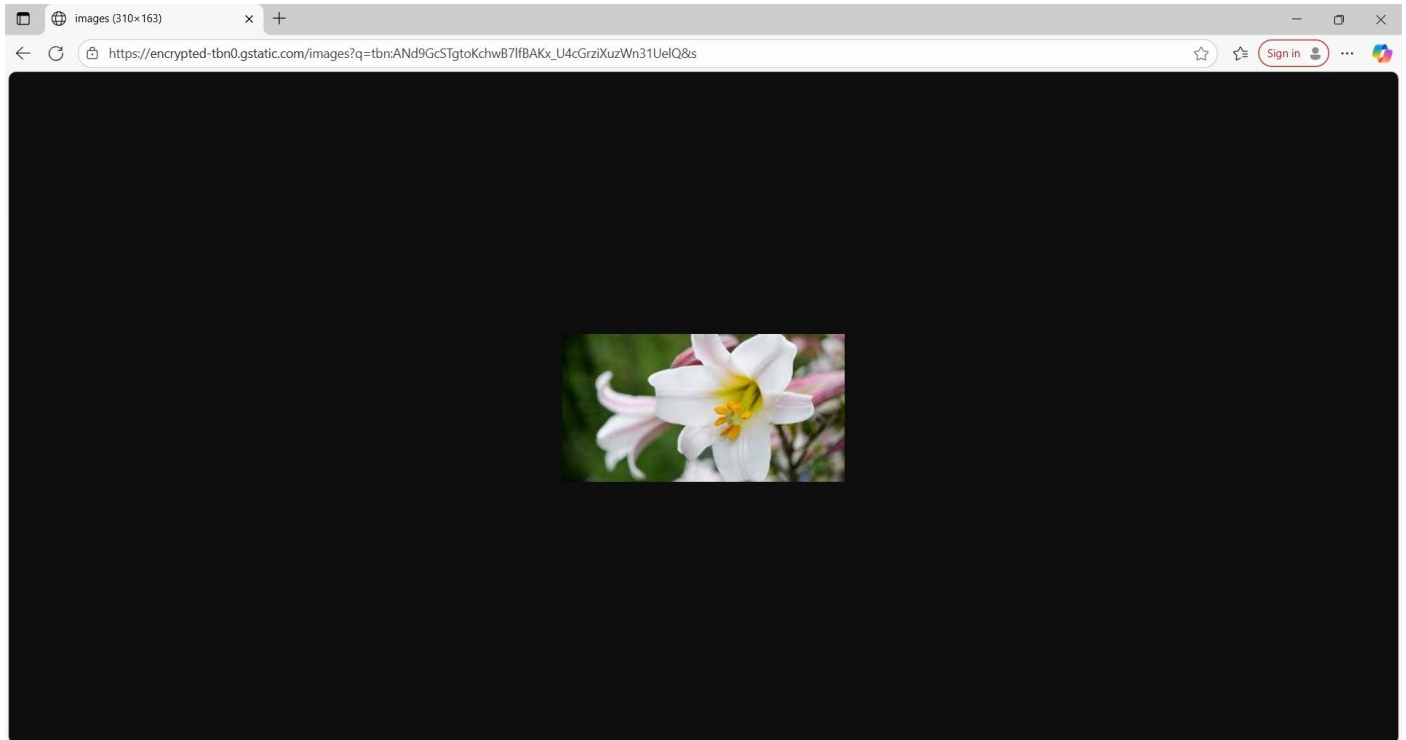
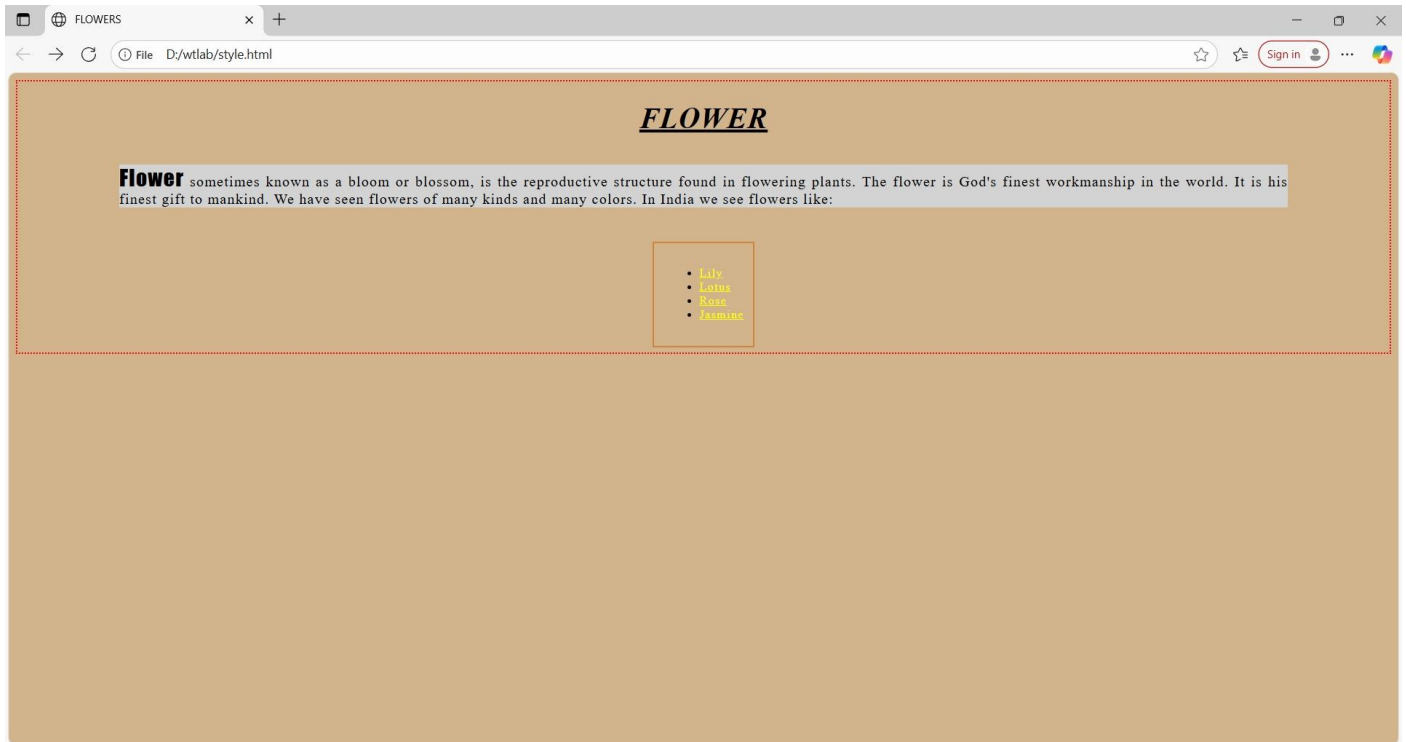
```
a:visited {  
    color: yellow;  
}
```

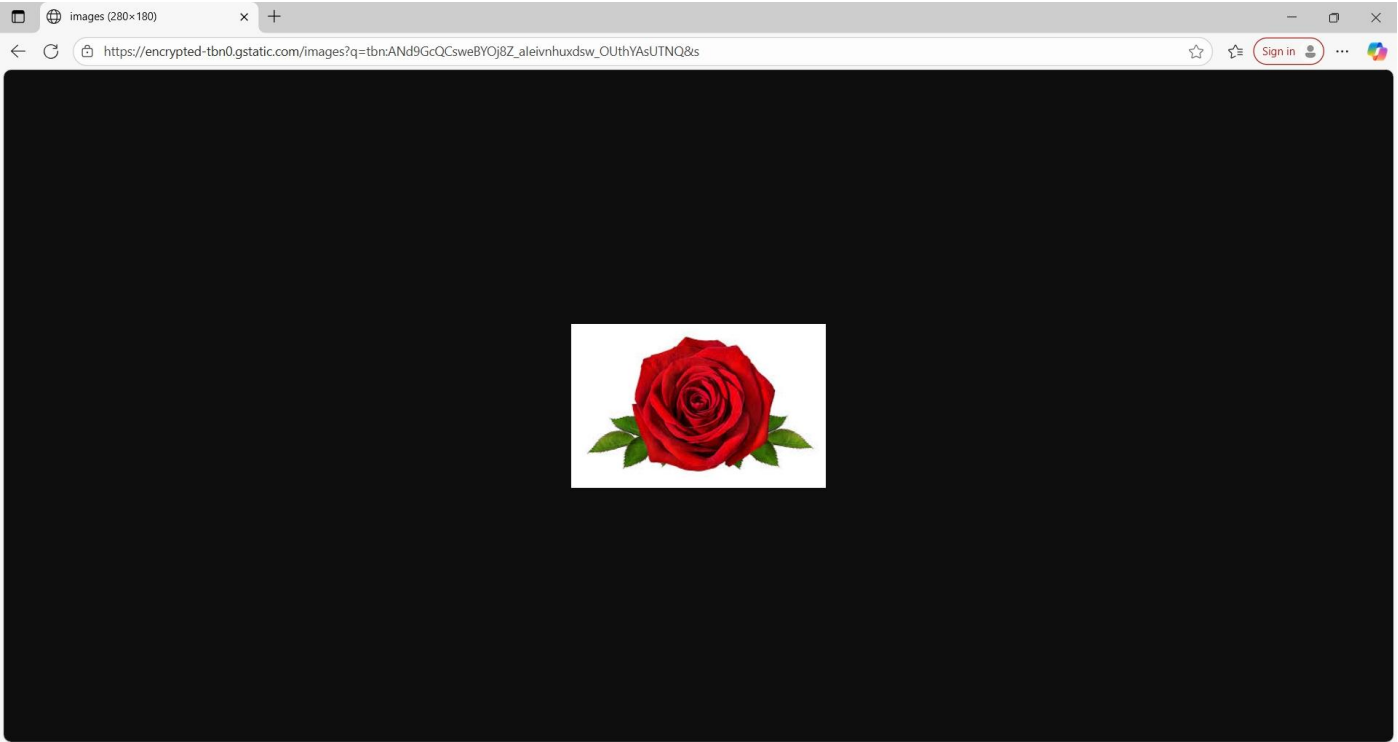
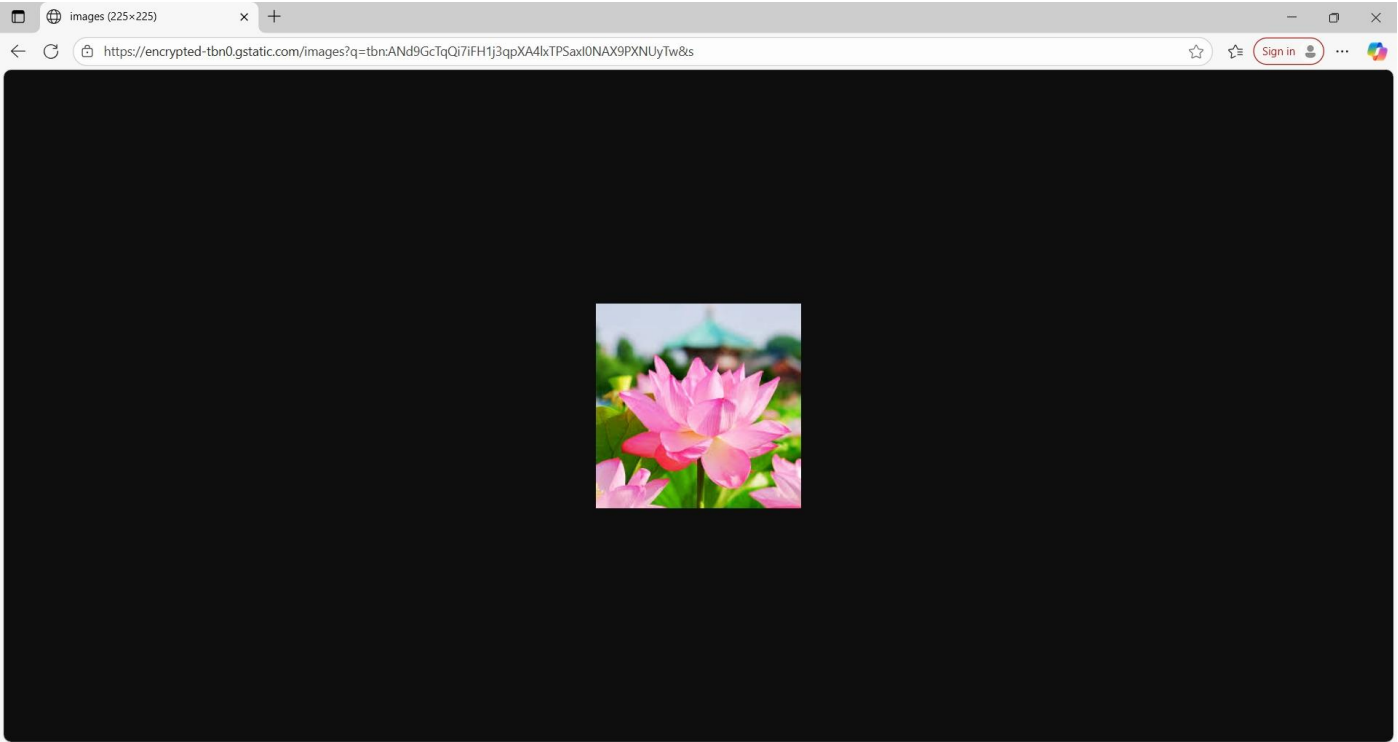
```
a:hover {  
    color: green;  
}
```

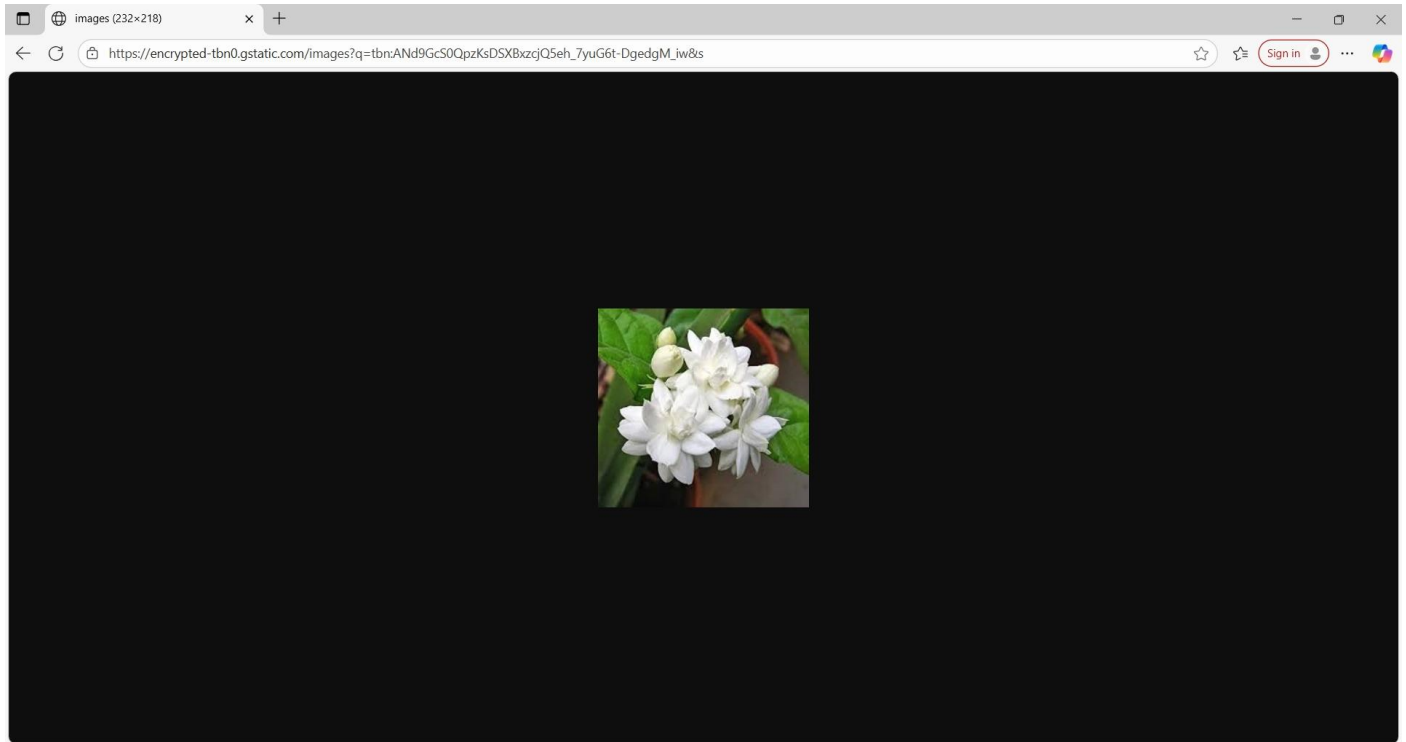
```
a:active {  
    color: blue;  
}
```

```
ul li {  
    font-size: small;  
}
```

# OUTPUT:







## **Result:**

The web page was successfully created using all three types of Cascading Style Sheets—Inline, Internal, and External. Each style type was applied correctly to demonstrate its effect on the layout, formatting, and visual presentation of the content.

**Aim:**

To develop a web page using Dynamic HTML (DHTML) that performs client-side validation of form controls such as text fields, radio buttons, checkboxes, and dropdowns before submission .

**Description:**

This experiment demonstrates how to perform client-side validation of web form controls using Dynamic HTML (DHTML), which combines HTML, CSS, and JavaScript. The validation ensures that required fields are filled before the form is submitted to the server. In this example, JavaScript is used to check whether the "Name" field is empty. If the field is left blank, an alert message is displayed and the form submission is prevented. This technique improves user experience by providing immediate feedback and reducing unnecessary server requests.

**Algorithm:**

**Step 1:** Start

**Step 2:** Create an HTML form with input fields (e.g., Name)

**Step 3:** Write a JavaScript function to validate the form inputs

**Step 4:** In the function, check if the required field is empty

**Step 5:** If the field is empty, display an alert message and return false to prevent form submission

**Step 6:** Link the validation function to the form's onsubmit event

**Step 7:** Create a separate HTML page (wlc.html) to display a welcome message upon successful submission

**Step 8:** Save and run the HTML file in a browser to test the validation

**Step 9:** Stop

**Program****control.html**

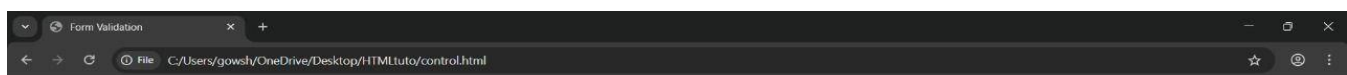
```
<!DOCTYPE html>
<html>
<head>
<title>Form Validation</title>
<script>
function validateForm() {
    var x = document.forms["myForm"]["fname"].value;
```

```
    if (x == null || x == "") {  
        alert("Name must be filled out");  
        return false;  
    }  
}  
</script>  
</head>  
<body>  
    <form name="myForm" action="wlc.html" onsubmit="return validateForm()" method="post">  
        Name: <input type="text" name="fname">  
        <input type="submit" value="Submit">  
    </form>  
</body>  
</html>
```

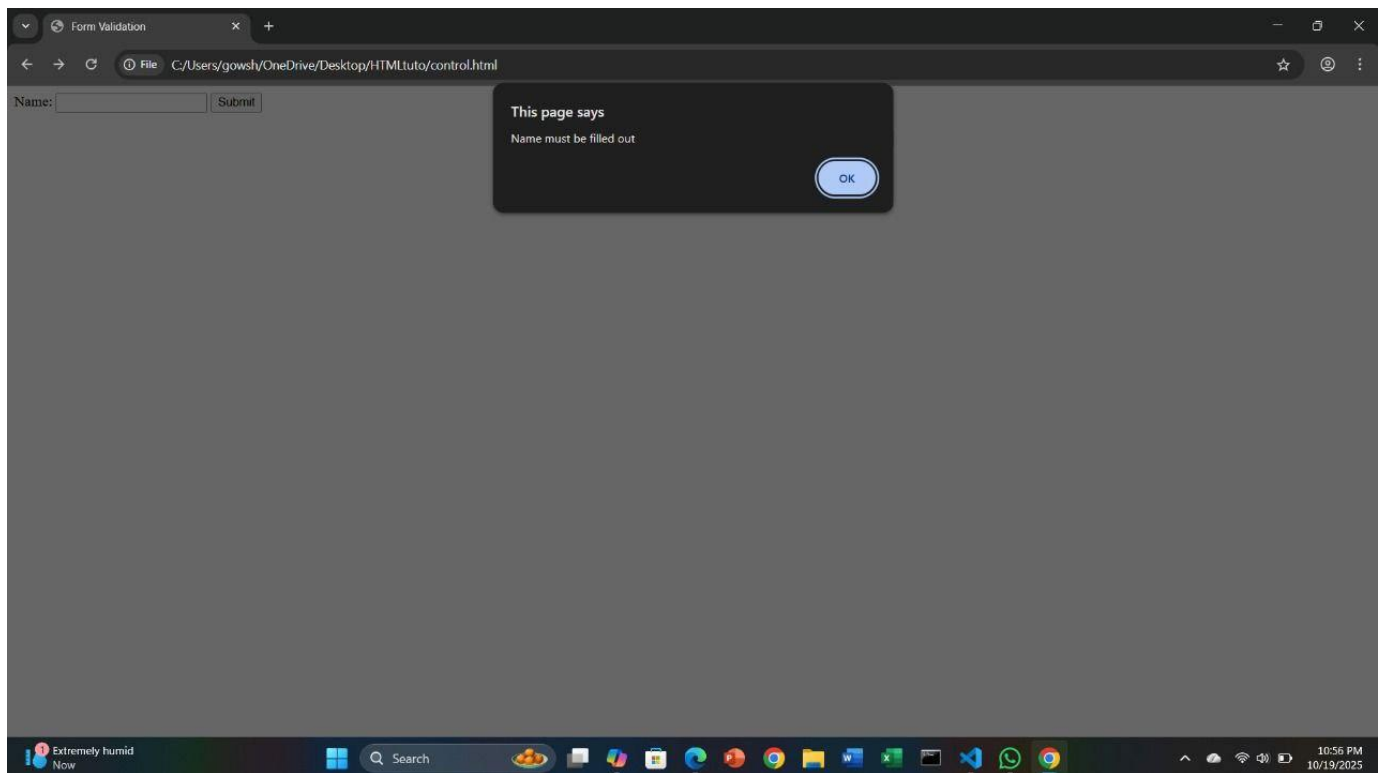
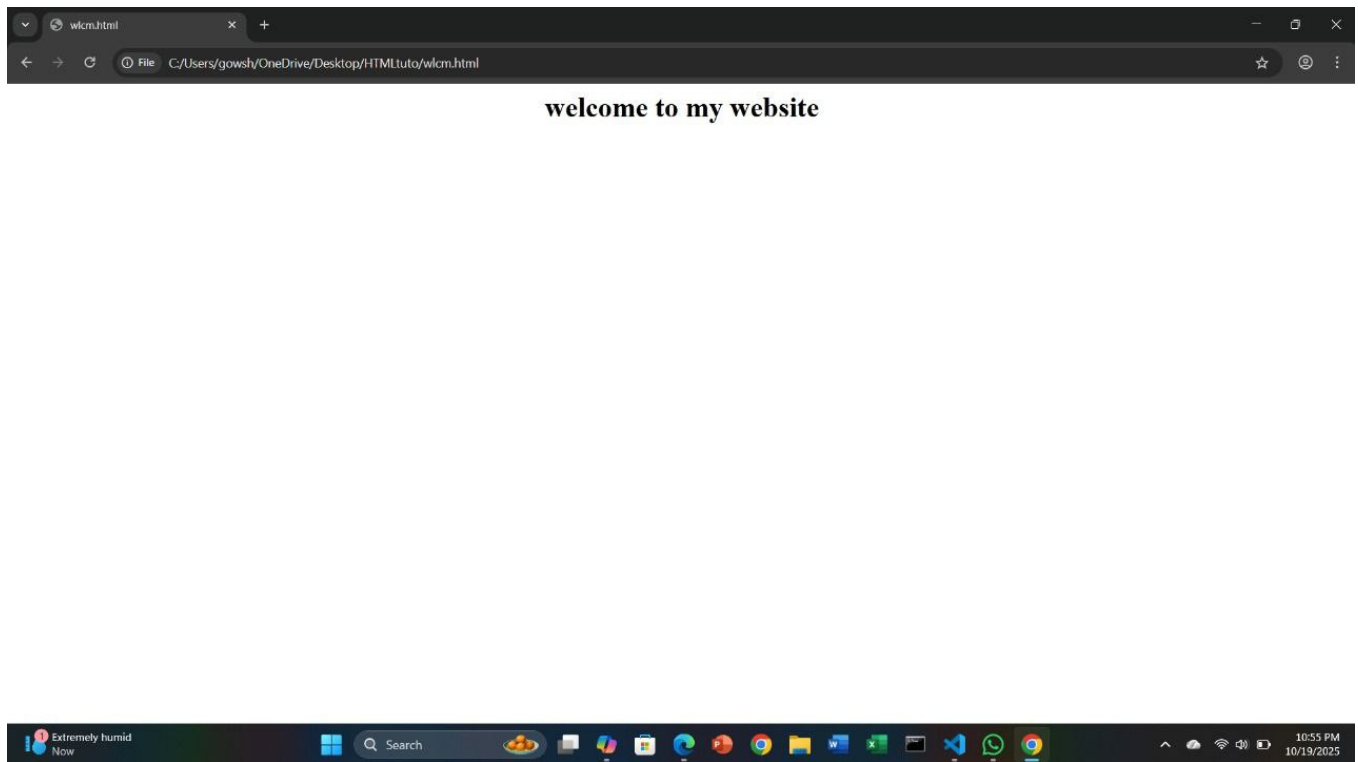
## wlc.html

```
<html>  
<body>  
<center>  
<h1>welcome to my website</h1>  
</center>  
</html>  
</body>
```

## OUTPUT:







## Result:

The web page was successfully developed using DHTML to perform client-side validation. The form correctly displayed an alert message when the "Name" field was left empty, preventing submission until valid input was provided.

**Aim:**

To write the steps for installing the Apache Tomcat web server in the system.

**Apache Tomcat HTTP Server:**

Apache Tomcat is a Java-capable HTTP server, which could execute special Java programs known as "Java Servlet" and "Java Server Pages (JSP)". Tomcat is an open-source project, under the "Apache Software Foundation" (which also provides the most used, open-

source, industrial-strength Apache HTTP Server). The mother site for Tomcat is <http://tomcat.apache.org>. Alternatively, you can find Tomcat via the Apache mother site @<http://www.apache.org>.

Tomcat was originally written by James Duncan Davison (then working in Sun) in 1998, based on an earlier Sun's server called Java Web Server (JWS). It began at version 3.0 after JWS 2.1 it was replaced. Sun subsequently made Tomcat open-source and gave it to Apache.

**The various Tomcat releases are:**

Tomcat 3.0 (1999): Reference Implementation (RI) for Servlet 2.2 and JSP 1.1.

Tomcat 4.1 (Sep 2002): RI for Servlet 2.3 and JSP 1.2.

Tomcat 5.0 (Dec 2003): RI for Servlet 2.4 and JSP 2.0.

Tomcat 6.0 (Feb 2007): RI for Servlet 2.5 and JSP 2.1.

Tomcat 7.0 (Jan 2011): RI for Servlet 3.0, JSP 2.2 and EL 2.2.

Tomcat 8.0 (Jun 2014): RI for Servlet 3.1, JSP 2.3, EL 3.0 and WebSocket 1.0. Tomcat 8.5 (June 2016) supports HTTP/2, OpenSSL, TLS virtual hosting and JASPIC 1.1.

Tomcat 9.0 (Jan 2018): RI for Servlet 4.0, JSP 2.3, EL 3.0, WebSocket 1.0, JASPIC 1.1.

Tomcat 10.0 (???):

**How to Install Tomcat and Get Started with Java Servlet Programming****2.1STEP 0: Create a Directory to Keep all your Works**

I shall assume that you have created a directory called "c:\my WebProject" (for Windows) or "-\my WebProject" (for Mac OS X) in your earlier exercises. Do it otherwise. This step is important; otherwise, you will be out-of-sync with this article and will not be able to find your files later.

## **2.2STEP 1: Download and Install Tomcat For Windows**

Goto <http://tomcat.apache.org> Under "Tomcat 9.0. (xx) Released", where {xx} is the latest update number =>Click "Download" Under "9.0. (xx)"=> Binary Distributions =>Core=> "zip" (e.g., "apache-tomcat-9.0.(xx).zip", about 11 MB).

UNZIP the downloaded file into your project directory "c:\my WebProject". Tomcat shall be unzipped into the directory "c:\my WebProject\apache-tomcat-9.0. (xx)". For EASE OF USE, "c:\my WebProject\tomcat". we shall shorten and rename this directory to "<c:\myWebProject\tomcat"

Take note of Your Tomcat Installed Directory. Hereafter, I shall refer to the Tomcat installed directory as <TOMCAT\_HOME>.

### **Tomcat's Sub-Directories**

Take a quick look at the Tomcat installed directory. It contains the these sub-directories:

bin: contains the binaries and scripts (e.g., startup.bat and shutdown.bat for Windows; startup.sh and shutdown.sh for Unixes and Mac OS X).

conf: contains the system-wide configuration files, such as server.xml, web.xml, and context.xml.

webapps: contains the webapps to be deployed. You can also place the WAR (Webapp Archive) file for deployment here.

lib: contains the Tomcat's system-wide library JAR files, accessible by all web apps. You could also place external JAR file (such as MySQL JDBC Driver) here.

logs: contains Tomcat's log files. You may need to check for error messages here.

work: Tomcat's working directory used by JSP, for JSP-to-Servlet conversion.

## **STEP 2: Create an Environment Variable JAVA HOME (For Windows)**

You need to create an environment variable (system variable available to all applications) called "JAVA\_HOME", and set it to your JDK installed directory.

Many Java applications (such as Tomcat) require the environment variable JAVA HOME to be set to the JDK installed directory.

To set the JAVA\_HOME environment variable:

First, find your JDK installed directory. For JDK 11, the default is "c:\Program Files\Java\jdk-11.0.{x}", where "{x} is the update number. Use your "File Explorer" to find this directory and take note of your update number {x}.

Check if JAVA\_HOME is already set. Start a CMD and issue:

set JAVA\_HOME

If you get a message "Environment variable JAVA\_HOME not defined", proceed to the next step.

If you get "JAVA\_HOME=C:\Program Files\Java\jdk-11.0.{x}", verify that it is set correctly to your JDK directory. If not, proceed to the next step.

To set the environment variable JAVA HOME in Windows 10:

Launch "Control Panel"=> (Optional) "System and Security"=> "System" =>Click "Advanced system settings" on the left pane.

Switch to "Advanced" tab=> Click "Environment Variables"

Under "System Variables" (the bottom pane)=> Click "New" (or Look for "JAVA HOME" and "Edit" if it is already set) => In "Variable Name", enter "JAVA\_HOME" In "Variable Value", enter your JDK installed directory you noted in Step 1. (In the latest Windows 10: you can push the "Browse Directory" button and navigate to the JDK installed directory to avoid typo error.)

To verify, RE-START a CMD (restart is needed to refresh the environment variables) and issue:

```
set JAVA_HOME
```

```
JAVA_HOME=c:\Program Files\Java\jdk-11.0. (x) < Verify that this is YOUR JDK installed directory
```

Notes: Windows' environment variables (such as JAVA\_HOME, PATH) are NOT case-sensitive.

### **STEP 3: Configure the Tomcat Server**

The Tomcat configuration files, in XML format, are located in the "conf" sub-directory of your

Tomcat installed directory, e.g. "c:\my WebProject\tomcat\conf" (for Windows) or "~/my Web Project/tomcat/conf" (for Mac OS X). The important configuration files are:

server.xml web.xml

context.xml

Make a BACKUP of the configuration files before you proceed!!!

#### **Step 3(a) "conf\server.xml" - Set the TCP Port Number**

Use a programming text editor (e.g., Sublime Text, Atom) to open the configuration file "server.xml".

The default TCP port number configured in Tomcat is 8080, you may choose any number between 1024 and 65535, which is not used by existing applications. We shall choose 9999 in this article. (For production server, you should use port 80, which is pre-assigned to

HTTP server as the default port number.)

Locate the following lines (around Line 69) that define the HTTP connector, and change port="8080" to port="9999".

<!-- A "Connector" represents an endpoint by which requests are received and responses are returned. Documentation at:

Java HTTP Connector: /docs/config/http.html

Java AJP Connector: /docs/config/ajp.html APR

(HTTP/AJP) Connector: /docs/apr.html

Define a non-SSL HTTP/1.1 Connector on port 8080

```
<Connector port="9999" protocol="HTTP/1.1"
```

```
    connectionTimeout="20000"
```

```
    redirectPort="8443"/>
```

### **Step 3(b) "conf\web.xml" - Enable Directory Listing**

Again, use a programming text editor to open the configuration file "web.xml".

We shall enable directory listing by changing "listings" from "false" to "true" for the "default" servlet. This is handy for test system, but not for production system for security.

Locate the following lines (around Line 108) that define the "default" servlet; and change the "listings" from "false" to "true".

```
<servlet>
```

```
<servlet-name>default</servlet-name>
```

```
<servlet-class>org.apache.catalina.servlets.DefaultServlet</servlet-class>
```

```
<init-param>
```

```
<param-name>debug</param-name>
```

```
<param-value>(0)</param-value>
```

```
</init-param>
```

```
<init-param>
```

```
<param-name>listings</param-name>
```

```
<param-value>>true</param-value>
```

```
</init-param>
```

```
<load-on-startup>1</load-on-startup>
```

```
</servlet>
```

### Step 3(c) "conf\context.xml" – Enabling Automatic Reload

We shall add the attribute reloadable="true" to the <Context> element to enable automatic reload after code changes. Again, this is handy for test system but not recommended for production, due to the overhead of detecting changes.

Locate the <Context> start element (around Line 19), and change it to <Context reloadable="true">.

```
<Context reloadable="true">
```

```
.....
```

```
.....
```

```
</Context>
```

### 2.5 STEP 4: Start Tomcat Server

The Tomcat's executable/programs and scripts are kept in the "bin" sub-directory of the Tomcat installed directory.

#### Step 4(a) Start Server

For Windows

I shall assume that Tomcat is installed in "c:\my WebProject\tomcat". Launch a CMD shell and issue:

```
C: // Change drive
```

```
cd \my WebProject\tomcat\bin // Change directory to your Tomcat's binary directory  
startup
```

```
// Run startup.bat to start tomcat server
```

#### Step 4(b) Start a Client to Access the Server

Start a browser (Firefox, Chrome) as an HTTP client. Issue URL "http://localhost:9999" to access the Tomcat server's welcome page. The hostname "localhost" (with IP address of 127.0.0.1) is meant for local loop-back testing within the same machine. For users on the other machines over the net, they have to use the server's IP address or DNS domain name in the form of "http://server Hostname Or IP Address:9999".

(Optional) Try issuing URL http://localhost:9999/examples to view the servlet and JSP examples. Try running some of the servlet examples.

## Step 4(c) Shutdown Server

For Windows

You can shutdown the Tomcat server by either:

Press Ctrl-C on the Tomcat console; OR

Run the "<TOMCAT\_HOME>\bin\shutdown.bat" script. Open a new "cmd" and issue:

e:// Change the current drive

cd \myWebProject\tomcat\bin// Change the directory to your Tomcat's binary directory  
shutdown// Run shutdown.bat to shut down the server

## Result:

Thus, the installation of the Apache Tomcat web server was executed successfully.

**Aim:**

To develop Java servlet programs that demonstrate form handling by invoking servlets from HTML forms and implement session tracking to maintain user-specific data across multiple requests.

**Description:**

This experiment demonstrates how to build a simple login application using Java Servlets and cookies for session tracking. The system includes three servlets—LoginServlet, ProfileServlet, and LogoutServlet—along with supporting HTML pages.

- The **LoginServlet** handles form submission, validates credentials, and sets a cookie to track the user session.
- The **ProfileServlet** retrieves the cookie to display personalized content if the user is logged in.
- The **LogoutServlet** clears the cookie to log the user out and redirects to the login page.

The web.xml deployment descriptor maps servlet names to their respective URLs and sets the welcome page. This setup illustrates how servlets interact with HTML forms and manage user sessions using cookies.

**Algorithm:**

**Step 1:** Start

**Step 2:** Create HTML pages (index.html, login.html, link.html) for navigation and form input

**Step 3:** Configure web.xml to map servlets and define the welcome page

**Step 4:** Implement LoginServlet to:

- Read form data
- Validate credentials
- Set a cookie with the username
- Redirect to ProfileServlet on success

**Step 5:** Implement ProfileServlet to:

- Read cookies from the request
- Display personalized content if the user is logged in
- Redirect to login page if no valid cookie is found

**Step 6:** Implement LogoutServlet to:

- Clear the cookie by setting its max age to zero
- Redirect to the login page

**Step 7:** Deploy the application on Apache Tomcat and test the login flow

**Step 8:** Stop



## Program:

### Index.html

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="ISO-8859-1">
    <title>Servlet Login Example</title>
  </head>
  <body>
    <h1>Welcome to Login App by Cookie</h1>
    <a href="login.html">Login</a> |
    <a href="LogoutServlet">Logout</a> |
    <a href="ProfileServlet">Profile</a>
  </body>
</html>
```

### Link.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Links</title>
  </head>
  <body>
    <a href="login.html">Login</a> |
    <a href="LogoutServlet">Logout</a> |
    <a href="ProfileServlet">Profile</a>
    <hr>
    <p>... other content goes here ...</p>
  </body>
</html>
```

### Login.html

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Login Page</title>
  </head>
  <body>
    <form action="LoginServlet" method="post">
```

```
Name: <input type="text" name="name"><br>
Password: <input type="password" name="password"><br>
<input type="submit" value="login">
</form>
</body>
</html>
```

## Web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://java.sun.com/xml/ns/javaee"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://java.sun.com/xml/ns/javaee
    http://java.sun.com/xml/ns/javaee/web-app_2_5.xsd"
  version="2.5">
  <!-- LoginServlet Configuration -->
  <servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.javatpoint.LoginServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/LoginServlet</url-pattern>
  </servlet-mapping>
  <!-- ProfileServlet Configuration -->
  <servlet>
    <servlet-name>ProfileServlet</servlet-name>
    <servlet-class>com.javatpoint.ProfileServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>ProfileServlet</servlet-name>
    <url-pattern>/ProfileServlet</url-pattern>
  </servlet-mapping>
  <!-- LogoutServlet Configuration -->
  <servlet>
    <servlet-name>LogoutServlet</servlet-name>
    <servlet-class>com.javatpoint.LogoutServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>LogoutServlet</servlet-name>
    <url-pattern>/LogoutServlet</url-pattern>
  </servlet-mapping>

  <!-- Add this section to define your default page -->
  <welcome-file-list>
```

```
<welcome-file>index.html</welcome-file>
</welcome-file-list>
```

```
</web-app>
```

### **LoginServlet.java**

```
package com.javatpoint;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
public class LoginServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        request.getRequestDispatcher("link.html").include(request, response);
        String name = request.getParameter("name");
        String password = request.getParameter("password");
        if (name == null || password == null) {
            out.print("Error: Please provide both username and password.");
            return;
        }
        if ("admin123".equals(password)) {
            out.print("You are successfully logged in!");
            out.print("<br>Welcome, " + name);
            Cookie ck = new Cookie("name", name);
            ck.setMaxAge(3600);
            response.addCookie(ck);
            response.sendRedirect("ProfileServlet");
        } else {
            out.print("Sorry, username or password error!");
            response.sendRedirect("login.html");
        }
        out.close();
    }
}
```

### **LogoutServlet.java**

```
package com.javatpoint;
```

```

import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
public class LogoutServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        request.getRequestDispatcher("link.html").include(request, response);
        Cookie ck = new Cookie("name", "");
        ck.setMaxAge(0);
        response.addCookie(ck);
        out.print("You are successfully logged out!");
        response.sendRedirect("login.html");
    }
}

```

### **ProfileServlet.java**

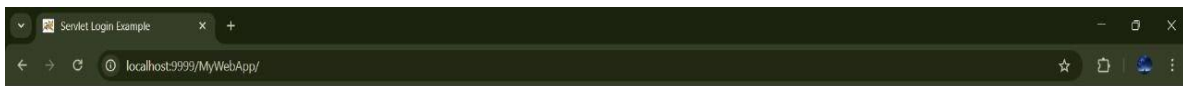
```

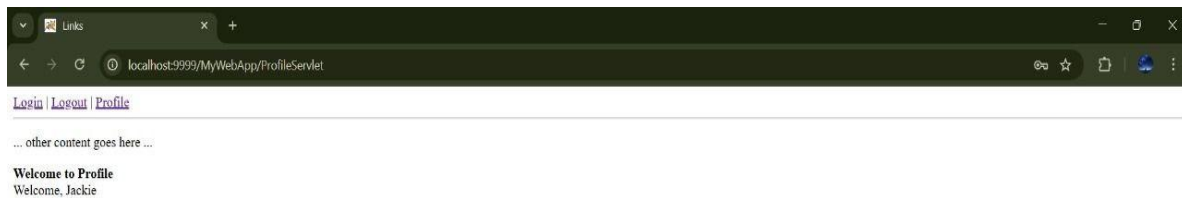
package com.javatpoint;
import java.io.IOException;
import java.io.PrintWriter;
import javax.servlet.ServletException;
import javax.servlet.http.Cookie;
import javax.servlet.http.HttpServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
public class ProfileServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {
        response.setContentType("text/html");
        PrintWriter out = response.getWriter();
        request.getRequestDispatcher("link.html").include(request, response);
        Cookie[] cookies = request.getCookies();
        String name = null;
        if (cookies != null) {
            for (Cookie cookie : cookies) {
                if ("name".equals(cookie.getName())) {
                    name = cookie.getValue();
                    break;
                }
            }
        }
    }
}

```

```
    }  
  }  
  if (name != null && !name.equals("")) {  
    out.print("<b>Welcome to Profile</b>");  
    out.print("<br>Welcome, " + name);  
  } else {  
    out.print("Please login first");  
    request.getRequestDispatcher("login.html").include(request, response);  
  }  
  out.close();  
}  
}
```

## OUTPUT:





## Result:

The login application was successfully developed using Java Servlets and cookies. The servlets correctly handled form submission, maintained user sessions through cookies, and enabled personalized profile access and secure logout functionality.

**Aim:**

To develop Java-based three-tier web applications using JavaServer Pages (JSP) and database connectivity for:

- Conducting online examinations
- Displaying student mark lists using data stored in a database server

**Description:**

This application allows students to take an online exam by entering their Roll No and Name. The system presents multiple-choice questions and calculates the score based on correct answers. The result is stored in a SQLite database using JDBC. The project demonstrates form handling, server-side scripting with JSP, and database connectivity in a three-tier architecture.

**Algorithm:**

**Step 1:** Start

**Step 2:** Create a login form to collect Roll No and Name

**Step 3:** Display exam questions using JSP

**Step 4:** Collect answers and calculate score

**Step 5:** Connect to SQLite database using JDBC

**Step 6:** Insert student details and score into the database

**Step 7:** Display result and thank-you message

**Step 8:** Stop

**Program:****index.html**

```
<html>
<head>
<title>Online Examination</title>
<style>
  body { text-align: center; font-family: Courier; color: blue; }
</style>
</head>
<body>
<h2>ONLINE EXAMINATION</h2>
<hr/>
<form action="exam.jsp" method="get">
```

```
Roll No: <input type="text" name="rno" required><br><br>
Name: <input type="text" name="name" required><br><br>
<input type="submit" value="Start Exam">
</form>
</body>
</html>
```

### exam.jsp

```
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<head>
<title>Exam Page</title>
<style>
body { color: blue; font-family: Courier; text-align: center; }
form { display: inline-block; text-align: left; }
</style>
</head>
<body>
<h2>ONLINE EXAMINATION</h2>
<hr/>
<%
String name = request.getParameter("name");
String rno = request.getParameter("rno");
if (name == null || rno == null || name.isEmpty() || rno.isEmpty()) {
    out.println("<p>Please enter your details first!</p>");
} else {
%>
<form action="result.jsp" method="post">
    <input type="hidden" name="name" value="<%=name%>">
    <input type="hidden" name="rno" value="<%=rno%>">

    <b>1. What is the capital of India?</b><br/>
    <input type="radio" name="q1" value="Delhi"> Delhi<br/>
    <input type="radio" name="q1" value="Mumbai"> Mumbai<br/>
    <input type="radio" name="q1" value="Chennai"> Chennai<br/><br/>

    <b>2. Which language is used for Android apps?</b><br/>
    <input type="radio" name="q2" value="Java"> Java<br/>
    <input type="radio" name="q2" value="Python"> Python<br/>
    <input type="radio" name="q2" value="C"> C<br/><br/>

    <b>3. Which protocol is used for web communication?</b><br/>
    <input type="radio" name="q3" value="HTTP"> HTTP<br/>
    <input type="radio" name="q3" value="FTP"> FTP<br/>
```



```
<input type="radio" name="q3" value="SMTP"> SMTP<br/><br/>
```

```
<input type="submit" value="Submit Answers">
</form>
<%
}
%>
</body>
</html>
```

### **result.jsp**

```
<%@ page import="java.sql.*" %>
<%@ page contentType="text/html; charset=UTF-8" %>
<html>
<head>
<title>Exam Result</title>
<style>
body { font-family: Courier; color: blue; text-align: center; }
</style>
</head>
<body>
<h2>EXAM RESULT</h2>
<hr/>

<%
try {
    // Get student info from exam.jsp
    String rno = request.getParameter("rno");
    String name = request.getParameter("name");

    String q1 = request.getParameter("q1");
    String q2 = request.getParameter("q2");
    String q3 = request.getParameter("q3");

    if(rno == null || name == null || q1 == null || q2 == null || q3 == null) {
        out.println("<p style='color:red'>Error: Some data is missing! Please go back and
submit the exam.</p>");
    } else {
        // Calculate score
        int score = 0;
        if("Delhi".equals(q1)) score++;
        if("Java".equals(q2)) score++;
        if("HTTP".equals(q3)) score++;
```

```

// Connect to SQLite
Class.forName("org.sqlite.JDBC");
Connection con = DriverManager.getConnection("jdbc:sqlite:C:/sqlite/examDB.db");

// Insert into table
PreparedStatement ps = con.prepareStatement(
    "INSERT INTO student_exam (rno, name, score) VALUES (?, ?, ?)");
ps.setInt(1, Integer.parseInt(rno));
ps.setString(2, name);
ps.setInt(3, score);
ps.executeUpdate();

ps.close();
con.close();

// Display result
out.println("<p>Thank you, <b>" + name + "</b>!</p>");
out.println("<p>Your Roll No: <b>" + rno + "</b></p>");
out.println("<p>Your Score: <b>" + score + " / 3</b></p>");
out.println("<br/><a href='index.html'>Take Exam Again</a>");
}
} catch(Exception e) {
    out.println("<p style='color:red'>Error: " + e.getMessage() + "</p>");
}
%>

</body>
</html>

```

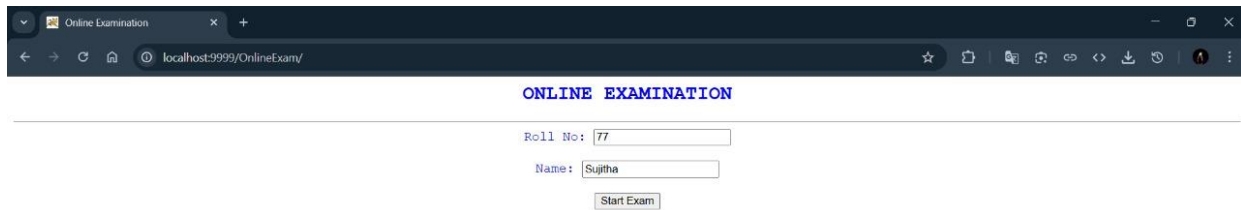
### **web.xml**

```

<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    version="3.1">
    <display-name>OnlineExam</display-name>
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>

```

## OUTPUT:



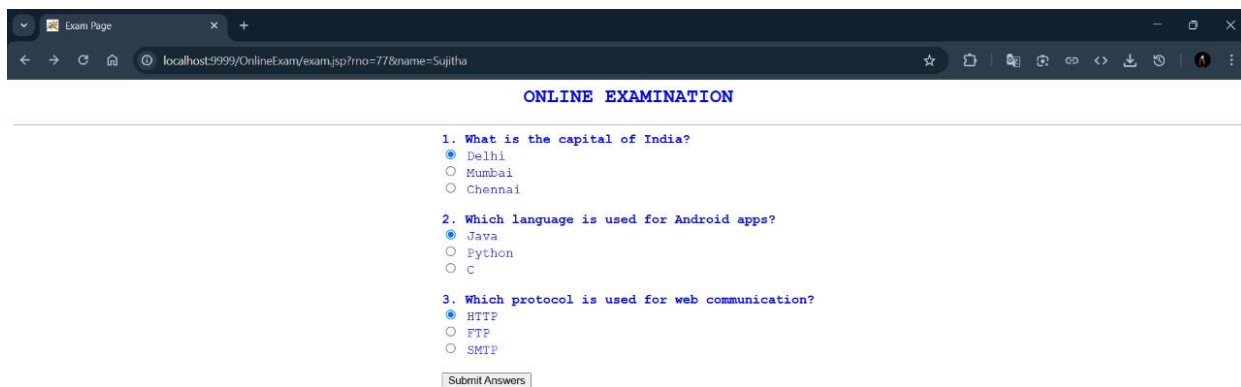
Online Examination

localhost:9999/OnlineExam/

ONLINE EXAMINATION

Roll No:

Name:



Exam Page

localhost:9999/OnlineExam/exam.jsp?no=77&name=Sujitha

ONLINE EXAMINATION

1. What is the capital of India?

- ☒ Delhi
- ☐ Mumbai
- ☐ Chennai

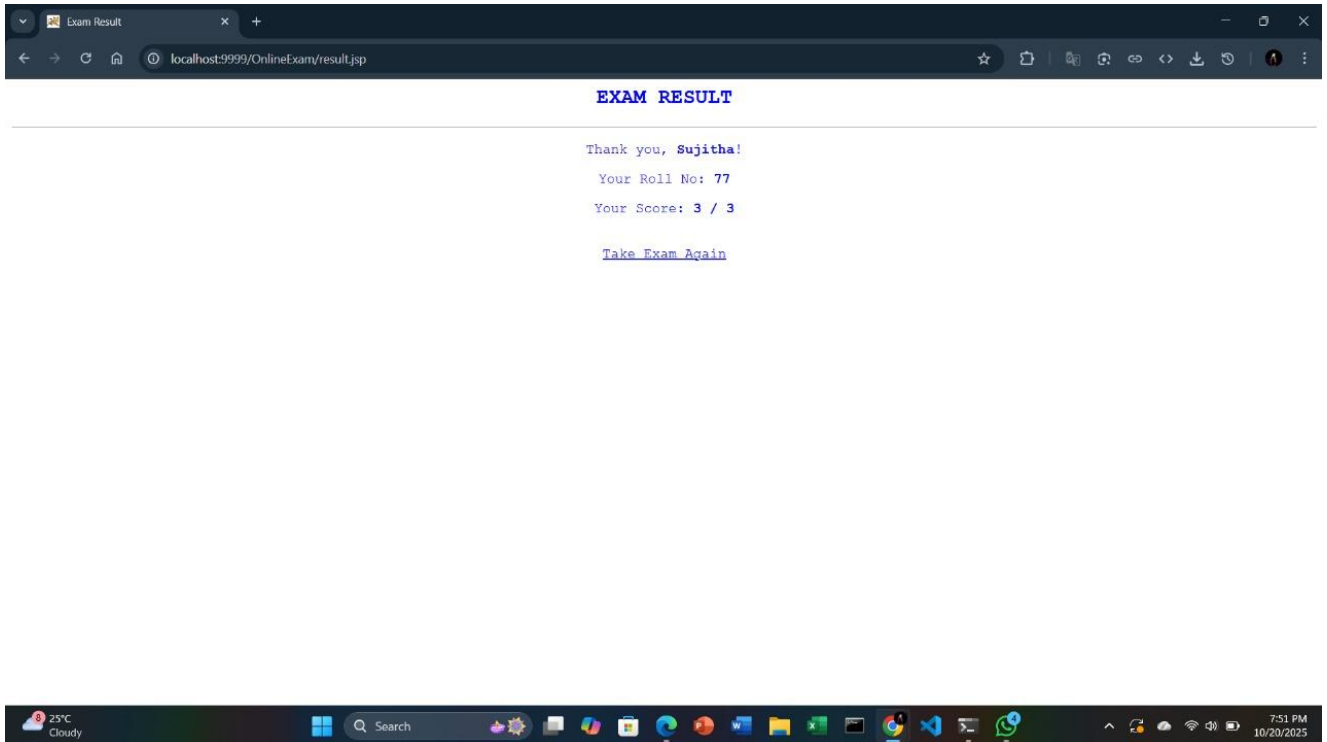
2. Which language is used for Android apps?

- ☒ Java
- ☐ Python
- ☐ C

3. Which protocol is used for web communication?

- ☒ HTTP
- ☐ FTP
- ☐ SMTP





## Result:

Thus, the three-tier applications for conducting online examination and displaying student mark list were successfully developed using JSP and database.

**Aim:**

To write programs using XML and XSLT to display CD catalog data in a formatted HTML table using transformation techniques.

**Description:**

This experiment demonstrates how to use XML for structured data representation and XSLT for transforming XML into a styled HTML output. The XML file (CDCatalog.xml) contains details of music CDs such as title, artist, country, company, price, and year. The XSL file (CDCatalog.xsl) defines a template to transform the XML data into an HTML table. It uses to loop through each CD entry and to organize the output by artist name. This technique is useful for presenting structured data in a readable format on web pages.

**Algorithm:**

**Step 1:** Start

**Step 2:** Create an XML file with CD catalog data

**Step 3:** Create an XSL file to define the transformation rules

**Step 4:** Link the XSL file to the XML using

**Step 5:** In the XSL file, use to match the root

**Step 6:** Use to iterate over CD entries

**Step 7:** Use to extract and display title and artist

**Step 8:** Apply to sort CDs by artist

**Step 9:** Save and open the XML file in a browser to view the transformed output

**Step 10:** Stop

**Program:****CDCatalog.xml**

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<?xml-stylesheet type="text/xsl" href="CDCatalog.xsl"?> <catalog>
  <cd>
    <title>Empire Burlesque</title>
    <artist>Bob Dylan</artist>
    <country>USA</country>
    <company>Columbia</company>
    <price>10.90</price>
```

```

    <year>1985</year>
</cd>
<cd>
  <title>Hide your heart</title>
  <artist>Bonnie Tyler</artist>
  <country>UK</country>
  <company>CBS Records</company>
  <price>9.90</price>
  <year>1988</year>
</cd>
<cd>
  <title>Greatest Hits</title>
  <artist>Dolly Parton</artist>
  <country>USA</country>
  <company>RCA</company>
  <price>9.90</price>
  <year>1982</year>
</cd>
</catalog>

```

### CDCatalog.xsl

```

<?xml version="1.0" encoding="ISO-8859-1"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:template match="/">
    <html>
      <body>
        <h2>My CD Collection</h2>
        <table border="1">
          <tr bgcolor="#9acd32">
            <th>Title</th>
            <th>Artist</th>
          </tr>
          <xsl:for-each select="catalog/cd">
            <xsl:sort select="artist"/>
            <tr>
              <td><xsl:value-of select="title"/></td>
              <td><xsl:value-of select="artist"/></td>
            </tr>
          </xsl:for-each>
        </table>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>

```

## OUTPUT:



### My CD Collection

| Title            | Artist       |
|------------------|--------------|
| Empire Burlesque | Bob Dylan    |
| Hide your heart  | Bonnie Tyler |
| Greatest Hits    | Dolly Parton |

## Result:

The XML and XSLT programs were successfully executed. The CD catalog data was displayed in a well-formatted HTML table, sorted by artist name using XSLT transformation.